```
Explicación línea por línea
<!DOCTYPE html> >  <!--Indica al navegador que es un documento de html-->
<html> <!--Inicia el documento html -->
    <head> <!--Cabeza del documento -->
        <title>Semana 6 - Concurrencia</title> <!--Titulo de la pestaña-->
        <script defer src="script.js"></script> <!--Carga el archivo
script.js en segundo plano y lo ejecuta solo cuando todo el HTML esté listo-
->
    </head> <!--Cierre del encabezado -->
    <body> <!--Abre el cuerpo del documento-->
        <h2>Comparación de ejecución secuencial y concurrente</h2> <!--
Titulo principal. Muestra un encabezado nivel 2-->
        <h3>Modo secuencial</h3> <!--Subtitulo de la primera sección.
Encabezado nivel 3-->
        <a href="secuencial.php" target="_blank"> <!--Es el enlace al
archivo secuencial.php y lo abre en una nueva pestaña del navegador -->
            <button>Ejecucion secuencial</button> <!--Crea un botón que dice
“Ejecucion secuencial” y al presionarlo abre secuencial.php-->
        </a> <!--Cierra el enlace -->
        <h3>Simulación de ejecución en paralelo</h3> <!--Subtitulo de la
primera sección. Encabezado nivel 3-->
        <button onclick="hacerParalelo()">Ejecutar en paralelo</button> <!--
Crea un botón que dice “Ejecutar en paralelo” y al hacer clic se ejecuta la
función “hacerParalelo()”-->
        <div id="resultado"></div> <!--Es una “cajita” en la página con
nombre resultado donde JavaScript puede mostrar mensajes dinámicamente. -->
    </body> <!--Cierra el cuerpo del documento-->
</html> <!--Finaliza el documento html -->
```

```
function hacerParalelo(){ //Se crea una función de nombre hacerParalelo()
    document.getElementById("resultado").innerHTML = "Iniciando ejecución
paralela..."; //En la "cajita" con nombre "resultado" se muestra el mensaje
"Iniciando ejecución paralela..."
    fetch("tarea1.php"); //Ejecuta el archivo tarea1.php en el servidor y
tráeme el resultado.
    fetch("tarea2.php"); //Ejecuta el archivo tarea2.php en el servidor y
tráeme el resultado.
    document.getElementById("resultado").innerHTML = "Tareas enviadas en
paralelo. Revisa la pestaña Network para ver la actividad."; //En la
"cajita" con nombre "resultado" se muestra el mensaje "Tareas enviadas en
paralelo. Revisa la pestaña Network para ver la actividad."
} //Cierre de la función
```

```php
<?php //Inicia php
$Tiempoinicio = microtime(true); //En la variable $Tiempoinicio se guarda el tiempo exacto en el que inicio la acción
sleep(3); //Hace que el programa se retrase el programa unos segundos
sleep(3);//Hace que el programa se retrase el programa unos segundos
$Tiempofin = microtime(true); //En la variable $Tiempofin se guarda el tiempo exacto en el que se terminó la acción
$duracion = $Tiempofin - $Tiempoinicio; //En la variable $duracion se guarda la diferencia entre el tiempo final y el tiempo de inicio
echo"Ejecucion secuencial completada en " .round($duracion,2). "segundos"; //Muestra el tiempo de ejecución total redondeado a 2 segundos
?>
```

```php
<?php //Inicia código php
$Tiempoinicio = microtime(true); //Guarda el momento exacto en el que empieza la tarea en la variable $inicio
sleep(3); //Hace que el programa se detenga por unos segundos. Espera 3 segundos antes de continuar
$Tiempofin = microtime(true); //Guarda la hora exacta cuando termina la tarea
$duracion = $Tiempofin - $Tiempoinicio; //En la variable $duracion se guardan los segundos que duro la ejecución
echo"Tarea 1 completada en " ,round($duracion,2). "segundos"; //Muestra el tiempo que tardo la ejecución en segundos y redondeando a dos dígitos
?>
```

```php
<?php //Inicia código php
$Tiempoinicio = microtime(true);//En la variable $Tiempoinicio guarda el momento exacto en el que inicia la tarea
sleep(3);//Hace que el programa se detenga por unos segundos. Espera tres segundos antes de continuar
$Tiempofin = microtime(true); //en la variable $Tiempofin guarda el momento exacto en el que se terminó la tarea
$duracion = $Tiempofin - $Tempoinicio; //En la variable de duración se guarda la diferencia entre el fin y el inicio de la tarea
echo"Tarea 2 completada en " .round($duracion,2). "segundos";//Se muestra la cantidad de segundos que tardo la tarea en ejecutarse
?>
```

Tabla de tiempos

Tiempo de ejecución	Tiempo obtenido
Secuencial	3.04 y 3.03 segundos
Múltiple	3.04 segundos

Respuestas investigadas

1. **¿Qué hace microtime(true)?**
   Devuelve el tiempo actual del sistema con precisión de microsegundos. Se usa para medir cuánto tarda en ejecutarse un programa o una tarea.
2. **¿Qué hace sleep(3)?**
   Detiene la ejecución del programa durante 3 segundos. En este código se usa para simular que el programa está haciendo una tarea que tarda tiempo.
3. **¿Qué representa la variable $inicio?**
   Es una variable que guarda el momento exacto en el que comienza la ejecución de la tarea.
4. **¿Cómo se calcula el tiempo total?**
   Se calcula restando el tiempo final menos el tiempo inicial.
5. **¿Qué hace round()?**
   La función round() redondea un número decimal. En el código se usa para mostrar el tiempo con solo 2 decimales.
6. **¿Qué hace fetch() en JavaScript?**
   fetch() se usa para enviar una petición al servidor y obtener datos o ejecutar un archivo.
7. **¿Por qué no se usa await?**
   Porque el objetivo es enviar las dos tareas al mismo tiempo. Si se usara await, el programa esperaría a que una tarea termine antes de empezar la otra, lo que sería secuencial y no paralelo.
8. **¿Por qué ambas tareas parecen ejecutarse al mismo tiempo?**
   Porque fetch() es asíncrono.
   Esto significa que el navegador envía ambas peticiones al servidor sin esperar a que la primera termine, por lo que se ejecutan casi al mismo tiempo.

Parte 2 – Pruebas y resultados

1. **¿PHP es multihilo?**
   No directamente. PHP normalmente se ejecuta en un solo hilo por cada petición. Sin embargo, el servidor web puede manejar muchas peticiones al mismo tiempo.
2. **¿Qué papel cumple apache o el servidor web?**
   El servidor web (como Apache) recibe las solicitudes de los usuarios y ejecuta los archivos PHP. También se encarga de manejar varias peticiones al mismo tiempo.
3. **¿Qué es concurrencia?**
   La concurrencia es la capacidad de manejar varias tareas al mismo tiempo, aunque no necesariamente se ejecuten exactamente al mismo instante.
4. **¿Es lo mismo concurrencia que paralelismo?**

No. Concurrencia: varias tareas avanzan al mismo tiempo. Paralelismo: varias tareas se ejecutan exactamente al mismo tiempo en diferentes procesadores.

5. **¿Qué pasaría si las tareas duran 10 segundos?**
   Si se ejecutan en paralelo, ambas terminarían aproximadamente en 10 segundos. Si fueran secuenciales, primero terminaría una (10 s) y luego la otra (10 s), por lo que el total sería 20 segundos.
6. **¿Qué pasaría si hubiera 50 usuarios al mismo tiempo?**
   El servidor tendría que manejar muchas peticiones simultáneamente.
   Dependiendo de la capacidad del servidor:

- Podría funcionar normalmente
- Volverse más lento si hay demasiadas solicitudes.

Conclusión personal

Al realizar esta práctica pude observar la diferencia entre la ejecución secuencial y la concurrente (paralelo) de tareas en un programa. Al ejecutar las tareas de manera secuencial, el tiempo total de las tareas se sumó, mientras que al ejecutarlas en paralelo utilizando el fetch(), ambas se realizaron simultáneamente, reduciendo el tiempo de espera percibido. Esta práctica demuestra cómo la ejecución de tareas de forma concurrente puede mejorar la eficiencia en la ejecución de diversos procesos.

Además, aprendí a mostrar mensajes de manera dinámica utilizando innerHTML y Java Script y conocí algunas otras funciones como el fetch que se utiliza para hacer una petición al servidor o el sleep que en este caso se utilizó para retrasar el proceso unos segundos.